SentryScan: Assignment 3.2 — Multi-Attribute Utility Optimization

Course: 21CSE525T — Agent Technology
Assignment: 3.2 — Multi-Attribute Utility Optimization & Infeasibility Filtering
Student: R Ganesh [RA2212704010031]
Repository: https://github.com/drgeekg/SentryScan
Target Branch: `assignment/3.2-utility-optimization`
Git Tag: `assignment-3.2`
Status: Completed & Verified

---

1. Executive Summary & Theoretical Context

Assignment 3.2 equips SentryScan with a formal **Multi-Attribute Utility Theory (MAUT)** optimization engine (Keeney & Raiffa, 1976; Russell & Norvig, Ch. 16), enabling rational decision-making across competing operational objectives.

1.1 Rational Agency & Utility Theory

A rational agent acts to maximize expected utility. In cyber vulnerability management, plans must balance five competing dimensions: 1. **Severity Coverage (C_{sev}):** Fraction of detected critical vulnerabilities addressed (benefit). 2. **Host Coverage (C_{hosts}):** Fraction of network fleet endpoints audited (benefit). 3. **Execution Cost ($Cost$):** Computational / bandwidth resource consumption (cost). 4. **Completion Time ($Time$):** Wall-clock duration in milliseconds (cost). 5. **Residual Risk ($Risk$):** Unmitigated exposure likelihood (cost).

---

2. Architecture & Implementation Specification

2.1 Normalized Multi-Attribute Utility Formula

For each candidate plan o, attributes are normalized to $[0, 1]$ across the candidate pool: $$U(o) = w_1\bar{C}*{sev} + *w_2\bar{C}*{hosts} - w_3\overline{Cost} - w_4\overline{Time} - w_5\overline{Risk}$$ where: $$\sum_{i=1}^5 w_i = 1.0, \quad w_i \ge 0$$

2.2 Infeasibility Filtering & Hard Constraints

Before utility computation, candidates exceeding operational thresholds are strictly filtered: - $\text{Cost} > \text{Budget}*{max} \implies U(o) = -\infty$ (Rejection: `COST_EXCEEDED`) - $\text{Time} > \text{Deadline}*{max} \implies U(o) = -\infty$ (Rejection: `TIME_EXCEEDED`) - $\text{BatchList} = \emptyset \implies U(o) = -\infty$ (Rejection: `EMPTY_BATCH_LIST`)

The rational choice is the feasible argmax: $$o^* = \arg\max_{o \in O_{feas}} U(o)$$ Ties are resolved deterministically by comparing $(C_{sev}, C_{hosts}, -Cost, \text{plan_id})$.

---

3. Rubric Compliance & Verification Matrix

Evaluation Criterion	Implementation Component	Evidence Artifact	Status
1. Normalized Multi-Attribute Utility Function	`src/sentryscan/optimizer/formula.py`: Implements $U(o)$ with bounded $[0, 1]$ normalization	`tests/unit/test_optimizer_formula.py` `evidence/candidate_comparison_table.md`	**PASSED**
2. Validated Weight Profiles	`src/sentryscan/optimizer/weights.py`: `UtilityWeights` model with $\sum w_i = 1.0$ validation, non-negativity constraint, and posture presets	`tests/unit/test_optimizer_formula.py` `evidence/weight_sensitivity_report.json`	**PASSED**
3. Hard Constraint & Infeasibility Filtering	`src/sentryscan/optimizer/engine.py`: Strict isolation of invalid candidates with explicit rejection reasons and $-\infty$ utility assignment	`tests/unit/test_optimizer_engine.py` `evidence/candidate_comparison_table.md`	**PASSED**
4. Feasible Argmax Selection & Deterministic Ties	`src/sentryscan/optimizer/engine.py`: Selection of $o^* = \arg\max_{o \in O_{feas}} U(o)$ with deterministic tie-breaking	`tests/unit/test_optimizer_engine.py` `demos/demo_assignment_3_2.py` Step 1	**PASSED**
5. Robust Edge & Boundary Handling	Safe handling of empty candidate sets, zero-range spans, non-finite values (NaN, Inf), and all-infeasible pools	`tests/unit/test_optimizer_edge_cases.py`	**PASSED**
6. Verification Against Independent Exhaustive Search	Exhaustive baseline verification checking 100% mathematical parity against brute-force evaluation	`tests/unit/test_optimizer_brute_force.py` `evidence/run_log.txt` Step 2	**PASSED**
7. Negotiation Seam Integration (`score_bid`)	Seamless refactoring of `src/sentryscan/negotiation/scoring.py` to delegate to `PlanOptimizer` preserving 100% backward compatibility	`tests/unit/test_negotiation.py` `tests/integration/test_multi_agent_coordination.py`	**PASSED**
8. Multi-Scenario Weight Sensitivity Analysis	Demonstrating rank-order and winner shifts across operational postures (balanced, risk-first, coverage-first, cost-constrained)	`evidence/weight_sensitivity_report.json` `demos/demo_assignment_3_2.py` Step 3	**PASSED**
9. Trace & Single-Writer Auditing	Sequential event log recording optimization requests, candidate scoring, and winner selection via `src/sentryscan/observability/recorder.py`	`evidence/trace_excerpt.jsonl`	**PASSED**

4. Execution Results & Empirical Evidence

4.1 Ten-Candidate Optimization Table (`evidence/candidate_comparison_table.md`)

Constraints: $\text{Cost} \le 50.0$, $\text{Time} \le 400.0$ ms. Weights: $w_{sev}=0.35, w_{hosts}=0.25, w_{cost}=0.15, w_{time}=0.15, w_{risk}=0.10$.

Plan ID	Proposer	C_{sev}	C_{hosts}	Cost	Time	Risk	Feasible?	Score $U(o)$	Decision
`plan-01-minimal`	`scout`	0.30	0.40	12.0	60ms	0.20	✅ Yes	`+0.1580`	Suboptimal
`plan-02-breadth`	`scout`	0.65	0.85	28.0	150ms	0.12	✅ Yes	`+0.3635`	Suboptimal
`plan-03-risk-heavy`	`triage`	0.90	0.60	32.0	180ms	0.06	✅ Yes	`+0.3840`	Suboptimal
`plan-04-balanced-opt`	`triage`	**0.92**	**0.88**	45.0	220ms	0.05	✅ Yes	`+0.4365`	**WINNER (argmax o^*)**
`plan-05-fast-triage`	`triage`	0.75	0.35	18.0	80ms	0.10	✅ Yes	`+0.3010`	Suboptimal
`plan-06-infeasible-cost`	`scout`	0.98	0.95	78.0	310ms	0.04	❌ No	`-\infty`	**REJECTED:** Cost Exceeded (78 > 50)
`plan-07-infeasible-time`	`triage`	0.80	0.70	40.0	450ms	0.08	❌ No	`-\infty`	**REJECTED:** Time Exceeded (450 > 400)
`plan-08-empty-batch`	`scout`	0.00	0.00	0.0	0ms	0.50	❌ No	`-\infty`	**REJECTED:** Empty Batch List
`plan-09-moderate`	`scout`	0.50	0.70	22.0	110ms	0.15	✅ Yes	`+0.2855`	Suboptimal
`plan-10-lean`	`triage`	0.82	0.50	24.0	130ms	0.07	✅ Yes	`+0.3495`	Suboptimal

4.2 Posture Sensitivity Analysis (`evidence/weight_sensitivity_report.json`)

- **Balanced Posture:** Winner = `plan-04-balanced-opt` ($U = +0.4365$).
- **Coverage-First Posture:** Winner shifts to `plan-02-breadth` due to highest $C_{hosts}=0.85$ under low cost.
- **Risk-First Posture:** Winner shifts to `plan-03-risk-heavy` with lowest residual risk (0.06).

---

5. Submission Package & Reproduction Instructions

5.1 Exact Submission Components

1. **Report Document:** `assignments/assignment_3_2/deliverables/SentryScan_Assignment_3_2_Report.pdf`
2. **Repository URL:** `https://github.com/drgeekg/SentryScan`
3. **Target Branch:** `assignment/3.2-utility-optimization`
4. **Immutable Git Tag:** `assignment-3.2`
5. **Packaged ZIP Archive:** `dist/SentryScan_assignment_3_2.zip`
6. **SHA-256 Checksum:** `dist/SentryScan_assignment_3_2.zip.sha256`

5.2 Verification Commands

```
# Run optimization demonstration
python demos/demo_assignment_3_2.py

# Execute optimizer test suite
pytest -v tests/unit/test_optimizer_formula.py tests/unit/test_optimizer_engine.py tests/unit/test_optimizer_brute_force
```